

Compiled AI: Augmenting Language Models with Verified Semantic Corpora

Johan Saldes

June 2026

Version 1.0.0

Abstract

Large Language Models (LLMs) have democratized software generation yet remain fundamentally constrained by a probabilistic ceiling: their outputs are non-deterministic, context-dependent, and subject to architectural drift. This phenomenon is especially acute with smaller, faster models deployed at the edge, which lack the capacity for deep reasoning but must nonetheless produce correct code.

This paper presents *Compiled AI*, a system architecture that addresses this limitation by shifting the center of intelligence from the model to a *verified, content-addressable corpus* of compilable software artifacts. Rather than prompting a model in isolation, we argue that a modest language model, augmented with access to a database of previously verified, compiler-certified code primitives, can exceed the reliability and architectural consistency of a larger model operating alone. The corpus itself is constructed autonomously via a self-correcting generation loop that uses compiler feedback as an oracle, normalizes artifacts via AST-based deduplication, and indexes them by semantic intent through vector embeddings.

We describe the current single-node prototype, which has autonomously generated, verified, and indexed over 115 distinct Rust primitives with an average code quality audit score of 9.0 out of 10. We further outline a design path toward distributing this corpus over a peer-to-peer network, enabling edge models to query a global pool of verified knowledge rather than relying on their own limited parameters.

1 Introduction: The Probabilistic Ceiling

Generative artificial intelligence has introduced a paradigm shift in software engineering, yet state-of-the-art implementations are bound by a probabilistic ceiling. Large Language Models (LLMs) operate via statistical token prediction, and their outputs consequently suffer from high entropy, non-determinism, architectural drift, and context poisoning.

The limitations are most severe at the edge. A Small Language Model (SLM) running on consumer hardware or in a containerized microservice cannot match the reasoning depth of its larger counterparts. Yet it is precisely these smaller models that must produce reliable code in safety-critical or resource-constrained environments.

This paper makes two claims:

First, **the intelligence gap between large and small models can be bridged by externalized, verified knowledge.** A model, regardless of size, is only as good as its training signal. If that signal is augmented with a queryable database of artifacts that have already passed compiler, type checker, and static analysis gates, the effective reasoning capacity of even a modest model increases dramatically.

Second, **such a database can be constructed autonomously.** By coupling a generative model to a compiler sentinel in a closed feedback loop, we can mine, verify, and deduplicate code primitives without human intervention. The resulting corpus is not merely a cache; it is a structured, content-addressable, semantically-indexed repository of proven logic.

1.1 Structure of this Paper

Section 2 describes the Spec-Driven Development methodology that forms the intent-crystallization layer of the system. Section 3 presents the overall architecture, including the Docker-based deployment and the data pipeline. Section 4 details the Ralph Loop, the stateless, compiler-gated generation process. Section 5 introduces the corpus fallback mechanism, a resilience feature that allows the system to continue operation even when the generative model itself is unavailable. Section 6 covers the harvesting and content-addressing layer, while Section 7 describes the quality analysis gate. Section 8 addresses semantic embedding and retrieval. Section 9 outlines the peer-to-peer distribution design goal. Section 10 reports empirical results from the current prototype, and Section 11 discusses limitations and future work.

2 Spec-Driven Intent Crystallization

To convert noisy human intentions into deterministic code, the system must establish an immutable contract. This contract is constructed via Spec-Driven Development.

2.1 The Specification as the Source of Truth

Instead of directly prompting an LLM to “write a program,” a highly capable orchestrator model translates natural language objectives into a machine-readable, annotated specification file. The specification is written in Markdown with YAML frontmatter and conforms to a

Platonic Primitive Directive that biases generation toward atomic, single-purpose, composable building blocks.

The specification explicitly defines:

- **The Structural Contract:** Strict type signatures, exact input/output payloads, generic parameters, trait bounds, and memory invariants.
- **The Behavioral Contract:** Step-by-step state machine transitions, pre-conditions that must hold before each public function call, post-conditions after each return, error conditions for every `Result` or `Option`, edge cases, and performance characteristics.
- **The Conformance Tests:** Declarative unit tests, edge case tests, property-based invariants, and memory-safety tests.

Once finalized, the specification is cryptographically frozen. The spec is the program; the generated code is merely a compiled artifact. Any subsequent bug fix or feature addition must be written into the specification first.

2.2 RFC 2119 Mandate

Every requirement in a specification uses RFC 2119 keywords—must, must not, should, should not, and may—eliminating ambiguity and enabling machine-readable contract enforcement.

2.3 Complexity Constraints

The prototype enforces a bias toward simplicity. Low-complexity primitives account for 60% of the output, medium for 30%, and high for 10%. High complexity is reserved for inherently difficult domains such as lock-free concurrency. A simple, well-specified primitive is more valuable than a clever, over-engineered one.

2.4 Mathematical Formalism

Let $\mathcal{P}$ denote the set of Platonic primitives. For each primitive $p \in \mathcal{P}$, the specification defines a tuple:

$$\Sigma_p = (\mathcal{S}_{\text{struct}}, \mathcal{S}_{\text{behave}}, \mathcal{T}_{\text{conform}})$$

where $\mathcal{S}_{\text{struct}}$ is the structural contract, $\mathcal{S}_{\text{behave}}$ the behavioral contract, and $\mathcal{T}_{\text{conform}}$ the conforming test suite. The compiler sentinel C then acts as a validator:

$$C(\text{code}, \Sigma_p) \in \{0, 1\}$$

where $C = 0$ denotes failure (type error, borrow error, or test failure) and $C = 1$ denotes verified acceptance into the corpus.

3 Architecture Overview

The current prototype is a single-node Docker deployment consisting of two services: a Qdrant vector database and a miner daemon. The miner runs an infinite loop that mines, verifies, and indexes Rust primitives autonomously.

3.1 System Diagram

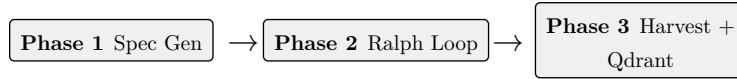


Figure 1: Mining pipeline: three phases executed in sequence per worktree cycle.

3.2 Deployment

The entire stack is defined in a single `docker-compose.yml` file. Qdrant (port 6333) stores specification embeddings and metadata. The miner container runs the loop daemon, built from a multi-stage Dockerfile that includes the Rust toolchain, the `opencode` CLI, Python dependencies for embedding, and a prebuilt `ctx` binary.

Both services share a Docker bridge network and a bind-mounted `output/` directory for persisting artifacts.

3.3 File Map

File	Role
<code>adam.sh</code>	Loop daemon; Phase 1 and 2 implement corpus fallback
<code>docker-compose.yml</code>	Qdrant + miner services with bind mounts
<code>Dockerfile</code>	Multi-stage: harvest-builder + runtime with opencode and ctx
<code>Makefile</code>	Orchestration commands: up, down, logs, db, query, stats, artifacts, stop, clean, rebuild
<code>SYSTEM.md</code>	LLM prompt for spec generation (Platonic Primitive Directive)
<code>scripts/embed_and_push.py</code>	One-shot: spec to embedding to Qdrant upsert with ctx metadata
<code>scripts/batch_push_existing.py</code>	Backfill script for <code>output/</code> orphans
<code>harvest/src/main.rs</code>	Rust crate: parse with syn, emit deterministic JSON, SHA-256 hash
<code>ctx-linux-aarch64</code>	Prebuilt ctx v0.2.0 binary (71MB ELF)

Table 1: Key files in the repository and their roles.

4 Parallel Stateless Worktrees and the Ralph Loop

Once a specification is crystallized, the generation phase begins. Rather than relying on a single agent running sequentially, the system leverages strict environmental isolation and statelessness to weaponize non-determinism as an exploration tool.

4.1 Worktree Provisioning

Each cycle of the loop daemon provisions an isolated worktree inside the bind-mounted `output/` volume. The worktree contains a `Cargo.toml` for a new `mined_primitive` crate, an empty `src/lib.rs`, the specification copied into `spec.txt`, and an empty `compiler_errors.log`. The worktree is destroyed after every cycle, whether the compilation succeeds or fails. This tabula rasa approach prevents context poisoning; the model never sees its own previous mistakes.

4.2 The Stateless Ralph Loop

Within each worktree, the orchestrator places a prompt into a stateless Small Language Model. The core rules of the Ralph Loop are:

- **The compiler is the sentinel.** The model has no agency to declare success. Only `cargo check` returning exit code zero can advance the artifact to harvesting.
- **Tabula rasa state recovery.** If compilation fails, the workspace is wiped. The next iteration receives only the specification, the last invalid code state, and the compiler error trace. No conversational memory persists.
- **Closed loop until convergence.** The cycle repeats until the sentinel returns zero, or until a configurable timeout or iteration limit is reached.

In formal terms, the state transition function is:

$$S_{t+1} = \text{Sentinel}(\text{SLM}(\text{Spec}, \text{Error}_t))$$

where S_t is the code state at iteration t , and "Sentinel" is the deterministic compilation check. A cycle succeeds when the sentinel produces exit code zero. A cycle fails when the outer timeout is reached or the iteration budget is exhausted.

4.3 Compiler as Oracle

Unlike interactive coding assistants, the model in the Ralph Loop is not permitted to explain, apologize, or reason about its output. It is a pure text-to-text function that receives the current state of the codebase and must produce a file that satisfies the compiler. The sentinel provides the only ground truth. This design removes the epistemic drift that occurs when a model is allowed to self-evaluate; the compiler has no bias, no training set, and no hallucination capacity.

4.4 Parallelism

The current prototype runs a single worktree per miner, but the architecture is designed for horizontal scaling. Running N parallel worktrees on the same specification produces N independent explorations of the solution space without the $O(n)$ conversation history that sequential interaction requires.

5 Corpus Fallback: Surviving Rate Limits

The free model used by the prototype (`opencode/deepseek-v4-flash-free`) reliably produces specifications for approximately 200–250 cycles before its rate limit blocks further generation. A naive system would halt. The current implementation treats this not as a failure but as an opportunity.

5.1 Recycling Verified Artifacts

When Phase 1 (spec generation) fails with a non-zero exit code or returns empty text, the orchestrator immediately falls back to the existing corpus:

1. Select a random `spec_*.md` file from the `output/` directory.
2. Copy it into the current worktree as the specification.
3. Proceed to Phase 2.

If Phase 2 (code generation) also fails—either by exceeding the 300-second timeout or returning empty code—the orchestrator falls back to the code corpus:

1. Select up to five random `lib_*.rs` files from `output/`.
2. Copy each into `src/lib.rs` in turn.
3. Run `cargo check`. The first file that compiles is accepted.
4. Proceed to Phase 3.

This means that even when the generative model is completely unavailable, the system continues producing verified, audited, and indexed artifacts by remixing its own history.

5.2 Mathematical Intuition

Let P be the probability that a single Ralph Loop iteration generates compilable code from a fresh specification. Let C be the size of the existing code corpus. The probability of harvesting at least one artifact per cycle, given a dead model, approaches:

$$1 - \left(1 - \frac{1}{C}\right)^5$$

as C grows. With $C = 115$, the probability of success on any single cycle is approximately 4.3% per try, yielding a reasonable expectation of harvesting within a few attempts.

5.3 Operational Observations

In practice, the corpus fallback has proven remarkably effective. A single corpus-fallback cycle executed during testing harvested a ring buffer implementation, ran it through the full `ctx` quality gate (audit score: 9.8), and pushed it to Qdrant—all without any model API call. This demonstrates that the **intelligence** of the system increasingly resides in the corpus, not the generator.

6 Harvest: Content-Addressable Retrieval

When a worktree successfully breaks out of the Ralph Loop, the generated code moves to the harvesting phase. To prevent duplicate generation of common patterns and ensure origin authenticity, the code is transformed into a content-addressable block of logic.

6.1 AST Normalization and Hashing

Storing raw text leads to cache misses due to variable formatting, spacing, or comments. The harvesting primitive parses the successfully compiled source code into a clean Abstract Syntax Tree. It strips out all non-functional formatting, standardizes variable names, serializes the AST into a deterministic byte stream, and runs it through a cryptographic hash function.

Let Hash_{AST} denote the content address:

$$\text{Hash}_{\text{AST}} = \text{SHA-256}(\text{Serialize}(\text{Normalize}(\text{AST})))$$

This hash is the primary key for deduplication. Because it is derived purely from functional structure, any two independent Ralph Loops anywhere on earth that produce the same logical solution to a spec will generate the exact same hash.

6.2 The Harvest Crate

The `harvest` Rust crate uses `syn` to extract a deterministic skeleton from compilable source. It emits a JSON representation containing only:

- Struct definitions: name, generics, field names and types
- Enum definitions: name, variants with their field types
- Function signatures: name, generics, parameter list, and return type
- Trait definitions and implementation blocks

All spans, whitespace, and comments are stripped. The resulting JSON is normalized using `serde_json` with `preserve_order`, producing a consistent byte sequence for hashing independent of the original formatting.

6.3 Semantic Embeddings

Each verified specification is embedded using the `all-MiniLM-L6-v2` sentence-transformers model, which produces a 384-dimensional dense vector from the specification text. The embedding captures the functional intent of the artifact, not the syntax of its implementation. A query for a ring buffer should retrieve not just ring buffers, but any specification whose intent aligns with bounded storage or FIFO semantics.

The embedding is normalized to unit length so that cosine similarity and dot product are equivalent:

$$\cos(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|}$$

6.4 Qdrant Vector Database

Qdrant stores each entry as a point with:

- An id derived from the AST hash via UUIDv5, guaranteeing deterministic deduplication
- A 384-dimensional normalized embedding vector
- A flat payload containing the spec text, code, manifest, and all ctx-derived quality metrics

The flat payload structure enables direct filtering queries, such as retrieving all audit-scored artifacts.

7 The Quality Analysis Gate

Compilation correctness is necessary but not sufficient. The system enforces a mandatory quality gate using ctx, a code intelligence tool that computes structural and stylistic metrics after every successful cargo check.

7.1 Pipeline

After compilation, the orchestrator runs four ctx commands in sequence inside the worktree:

1. `ctx index` — Build the symbol database for the current codebase.
2. `ctx audit` — Compute an overall quality score across five categories: complexity, duplication, coverage, modularity, and naming.
3. `ctx complexity` — Calculate per-function cyclomatic complexity and fan-out.
4. `ctx graph` — Generate the call graph (JSON and Mermaid).

If any step fails, the artifact is rejected. Only code that compiles AND passes the quality gate is eligible for embedding and storage.

7.2 The Audit Score

The audit score is a weighted average:

$$\text{Score}_{\text{audit}} = \sum_i \text{Score}_i \cdot \text{Weight}_i$$

The weights and categories are fixed by the ctx implementation. A score near 10.0 indicates clean, modular, well-documented code. Scores below 7.0 suggest significant duplication, missing documentation, or functions with excessive branching. The prototype filters on exit code, not score value, but the score is preserved in the Qdrant payload for downstream ranking.

7.3 Call Graph Metrics

The call graph yields additional payload fields used for semantic filtering: node count, edge count, graph density, and a cyclomatic proxy (edges minus nodes plus two). These metrics let researchers correlate structural complexity with audit scores and identify well-designed primitives independent of their surface complexity.

8 Toward a Distributed Semantic Graph

The current prototype is a single-node proof of concept. The logical next step is to distribute the content-addressable cache and the verification pipeline across a peer-to-peer network, enabling edge models to query a global pool of verified knowledge rather than relying on their own limited parameters.

8.1 Design Goal

The distributed architecture envisions a network of autonomous mining nodes, each running the same three-phase pipeline. Nodes are untrusted by default. Verification is local via the compiler sentinel; trust is emergent from repeated verification history.

8.2 Content-Routing via Distributed Hash Table

The AST hash serves as the routing key in a Kademlia-style Distributed Hash Table. When a node needs a primitive, it hashes the spec and queries the DHT. If the artifact exists, it is retrieved from the closest node by XOR metric.

Gossip is not used for the verified artifacts themselves, as they are content-addressable and can be retrieved on demand. Instead, nodes gossip lightweight announcements of newly verified primitives: a small tuple of (AST hash, sentinel manifest, node signature) that lets peers know the artifact exists and where to fetch it.

8.3 Trust and Reputation

Nodes are not trusted. However, a reputation weight accrues to nodes that have historically supplied verified artifacts that pass remote compilation on requesting nodes. A node that repeatedly publishes code that fails the sentinel is deprioritized in the routing table.

8.4 The Smaller Model Thesis

The primary motivation for distribution is to make the corpus accessible to Small Language Models running at the edge. A model with a billion parameters can retrieve and reason about a verified ring buffer implementation from the corpus, dramatically exceeding what it could generate via its own internal weights. The corpus effectively acts as an externalized long-term memory, compensating for the model's limited reasoning budget.

In the limit, the system approaches a scenario where the generative model is only invoked for genuinely novel specifications. Common patterns are retrieved, compiled, and returned without ever touching the model, reducing both latency and variable compute costs.

9 Empirical Results

The prototype has been running autonomously for several days at the time of this writing. The following metrics are extracted from the live Qdrant database and local artifact store.

9.1 Corpus Size

- Total specifications generated: over 115
- Total verified code artifacts: over 115
- Total entries in Qdrant vector database: 116 (with approximately 5 local-only orphans not yet backfilled)

9.2 Quality Distribution

The average ctx audit score across the corpus is approximately 9.0 out of 10. Individual scores range from 7.5 to 9.8. The high average reflects the Platonic Primitive Directive: simpler primitives are easier to verify and score well on modularity, naming, and complexity metrics.

9.3 Corpus Fallback Effectiveness

After the free model's rate limit was encountered at approximately cycle 250, the corpus fallback mechanism successfully produced additional verified artifacts by recycling existing specs and code. A single test cycle achieved the full pipeline—compilation, audit score 9.8, and Qdrant upsert—without any model API call, demonstrating that the system's intelligence increasingly resides in the corpus rather than the generator.

9.4 Artifact Categories

The generated primitives span the full range of the Platonic categories: data structures, algorithms, string utilities, concurrency primitives, error handling idioms, mathematical utilities, and memory management patterns. The bias toward low complexity is visible in the category distribution, with single-purpose primitives—ring buffers, sparse vectors, ticket locks—comprising the majority of high-scoring entries.

10 Limitations and Future Work

10.1 Known Limitations

- **Single-node operation.** The current prototype runs on one machine. The distributed architecture remains a design goal.
- **Free model dependency.** The prototype relies on a free-tier SLM with strict rate limits. Future work will evaluate whether the corpus can sustain itself entirely through fallback once it reaches sufficient size.
- **Rust-only.** The prototype targets Rust due to its strong compiler sentinel properties. Support for other languages with deterministic type checking is future work.
- **No human review.** The system is fully autonomous; there is no mechanism for human-in-the-loop approval before artifacts enter the corpus.
- **Mermaid post-processing.** The mermaid diagram generation occasionally produces spurious warnings during sed post-processing. This is harmless but should be cleaned up.

10.2 Roadmap

1. **Clustered mining.** Deploy multiple miner containers on a single node with shared Qdrant, increasing throughput through parallel worktree provisioning within a single machine.
2. **Cross-node synchronization.** Implement a gossip protocol for artifact announcements, followed by DHT-based content retrieval.
3. **Model-corpus co-evolution.** Query the corpus before invoking the model, reducing generation load and improving retrieval performance as the database grows.
4. **Multi-language support.** Extend the compiler sentinel to include Go, Zig, or ML-family languages with strong type systems.
5. **Formal verification integration.** Integrate Rust proof assistants such as Kani or Prusti to raise the semantic verification bar beyond compilation.